COP4533 Programming Project Guide

Below are some suggestions to guide you as you work on the project and prepare your report.

1 Algorithm Description

The purpose of the algorithm description is to communicate your approach as effectively as possible. A long narrative describing each line of your code is not ideal. Instead, aim for a **concise, high-level summary**, avoiding unnecessary implementation details.

Including pseudocode can help clarify your approach, but keep it **brief and language-independent**. For example, `make_tuple` is specific to C++ and unnecessary for algorithm explanation. Try comparing with pseudocode you come across in books.

Avoid ambiguous instructions such as "pick the best option." **Specify what is being compared and how the choice is made.**

2 Correctness Proof

When reviewing your proof, ask yourself if it could **"prove" an incorrect algorithm**.

Below are two incorrect proofs using the interval scheduling problem from the report template.

The algorithm processes activities in order of earliest finish time, selecting each activity compatible with those already selected.

2.1 Incorrect Proof by Induction

Let $a_1, \ldots, a_n$ be the activities in processing order. Let G_k be the set selected after processing the first k activities.
Base case ($k = 1$): Selecting the only activity is optimal.
Inductive hypothesis: Assume G_k is a maximum-size compatible subset of the first k activities.
Inductive step: Consider a_{k+1}.

1. If a_{k+1} is compatible with G_k, select it. The solution size increases by one, which is the largest possible increase.
2. Otherwise, a_{k+1} cannot be selected. Thus, keeping G_k is optimal.

Therefore, by induction, G_n is optimal.

The mistake: In case 2, we have only shown that the new activity cannot be added to the current selection. We have not shown that **replacing previously selected activities** cannot give a better solution.

Notice that this argument **never uses the earliest-finish-time ordering**. It could also incorrectly "prove" an earliest-start-time strategy.

2.2 Incorrect Exchange Argument

Let g be the activity with earliest finish time, and let O be an optimal solution. Write $f(a)$ for the finish time of activity a.
If $g \notin O$, choose any activity $o \in O$. Since g finishes earliest,

$$f(g) \leq f(o).$$

Replace o with g:

$$O' = (O \setminus \{o\}) \cup \{g\}.$$

Since g finishes no later than o, the replacement preserves compatibility. Also, $|O'| = |O|$, so O' is optimal. Repeating this exchange gives the greedy solution.

The mistake: The replacement does **not necessarily preserve compatibility**. For example, let $g = [0, 2]$ and $O = \{[1, 3], [3, 4]\}$. Replacing $[3, 4]$ leaves two overlapping activities.

The choice of the activity being replaced matters. "Exchange an activity with the greedy choice" is not enough: **specify which activity and check compatibility with those that remain**.

3 Experimental Study

The goal of the experimental study is to visualize and understand how the algorithms behave as input size increases.

- Pick input sizes that **go high enough** to show the growth in running time.
- Use **enough input sizes across the range**. Plot their actual numerical values so that spacing is represented correctly.
- Describe how you generated the inputs and measured running time. **Explain your observations** alongside the plots.

4 Presentation

Keep your report concise and to the point while covering everything required.

Use headings and subheadings that divide the content in a sensible way. A single large paragraph explaining the algorithm, analyzing its time complexity and proving correctness is not easy to read. **Effective use of headings and spacing** makes the report easier to follow.

Keep your code clean and easy to read. Pick sensible variable names. Keep the logic and flow as simple as possible while following the required algorithm and constraints.

5 Bonus Points

For bonus points, the main emphasis is on **report presentation quality**. Put extra effort into making your report **well-formatted, structured, concise and easy to read**. Your code should also be concise, well-structured and well-commented so that the control-flow and algorithmic approach is easy to understand just by reading it.

Come up with creative ways to provide **deeper insights** into your algorithms. For example, you could improve your experimental study by using multiple samples and error bands, comparing measured runtime with theoretical growth using carefully picked constants, or using heatmaps and contour plots to explore multiple variables. **The specifics of how you improve the study are up to you.**

> **Longer and more is not always better. It is often worse.** Use your judgement about what to include. Extra plots and explanations should add meaningful insights and make your work easier to understand.